

---

# 目的ファーストのハーネス設計

ハーネスの変更容易性を高めるための優先順位

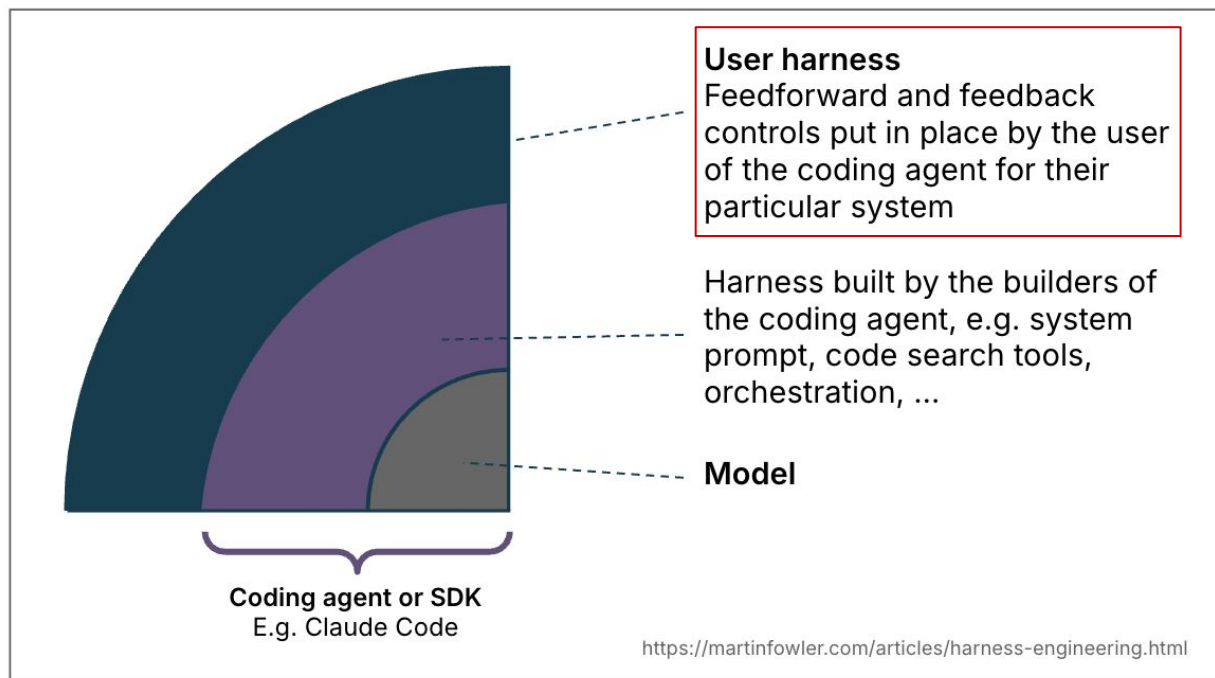
Findy - Harness Engineering入門 (2026-04-22)

Gota(@gota\_bar)

---

# 今日の「ハーネス」はユーザーハーネスの話

今日はコーディングエージェントの外側のユーザーハーネスの話をしてします。  
オーケストレーションの話はしません



# 自己紹介

Gota (@gota\_bar)



## 所属

Agentic AI エンジニア & アナリティクスエンジニア

## やってること

AI エージェント開発 / 小売向けデータプロダクト / データ整備

## 好きなエージェント

pi ... 超シンプルなTerminal型エージェント. システムプロンプトを人間が容易に読める



cc-sdd

Public



Turn approved specs into long-running autonomous implementation. A minimal, adaptable SDD harness with Agent Skills for Claude Code, Codex, Cursor, Copilot, Windsurf, OpenCode, Gemini CLI, and Anti...

TypeScript

☆ 3.2k

🔗 241

## cc-sdd v3.0 (2026/4 update)

- 境界をファーストクラス化
- 長時間の自律実装：specのTDD実装 ↔ レビューのClosed Loop
- Slash commandをAgent Skillsに変更

# 現場でよく聞く話

## 段階は違うが、根っこは一つ「判断基準が明文化されていない」

### ハーネスがない

- 一人一人バラバラに指示
- Agentが出すコードの品質が安定しない

### 作り始めた

- ルール増えすぎ、Skillに切り出しても読まれない / 効かない
- CLAUDE.md 1000行超、指示の半分が無視される

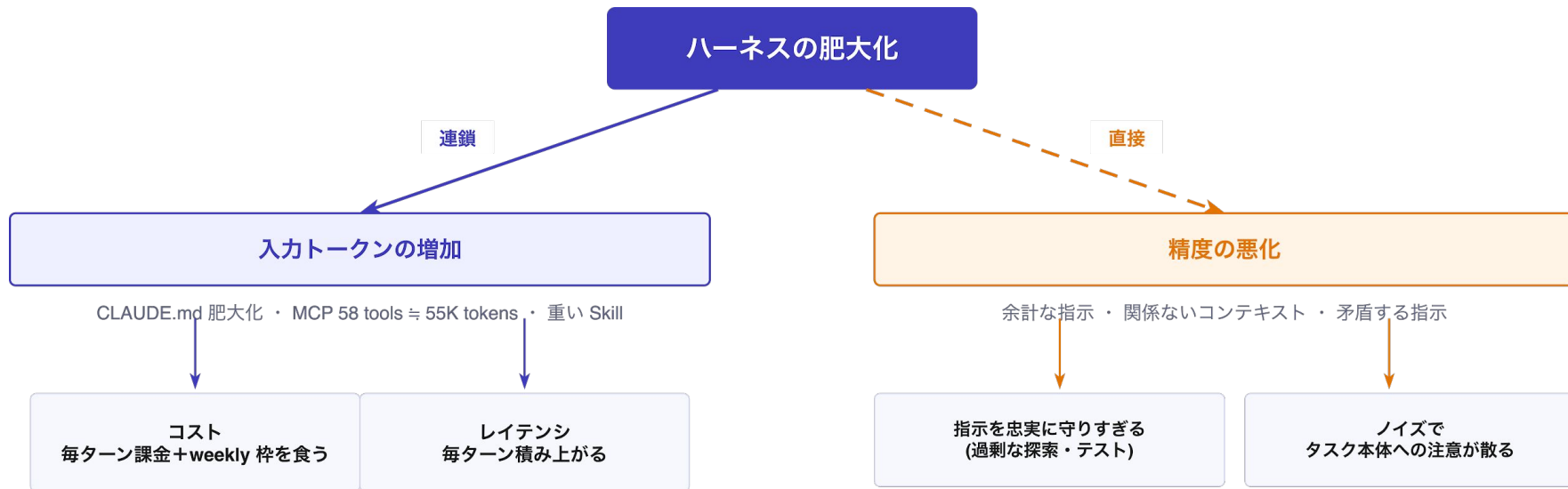
### 運用中

- モデルが変わっても、旧モデル向けの指示が剥がせない
- 半年前の設定、消していいか / 更新していいかわからない
- オーケストレーションがドリフトする

足せても削れず、ハーネスは肥大化し、やがて変更できなくなる

# 判断基準がないまま太ると何が起きるか

トークンを増やしてコストが増加。加えて指示の内容が直接 精度を下げる。



# ハーネスを分解する

HOW はたくさんあるが、業務目的と状況で最適な選択肢は変わる  
→業務目的から逆算する必要がある

|                              | Guide（事前に導く）                                                                                                                                                                         | Sensor（事後に検知）                                                                                                                                                               |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Computational</b><br>決定論的 | <p>軽</p> <p>不正な状態を書けなくする      <b>型 / schema / ADT</b></p> <p>層を越えた import を封じる      <b>モジュール境界 / 依存方向</b></p> <p>正しい形から書き始める      <b>scaffolding / codemod</b></p> <p>↓</p> <p>重</p> | <p>軽</p> <p>IDE 毎ストローク      <b>type check</b></p> <p>on save / pre-commit      <b>lint / unit test</b></p> <p>CI / PR 時      <b>integration / e2e</b></p> <p>↓</p> <p>重</p> |
| <b>Inferential</b><br>LLM 推論 | <p>軽</p> <p>常駐・全ターン課金      <b>CLAUDE.md / AGENTS.md</b></p> <p>呼び出し時のみ      <b>Skill（指示系）</b></p> <p>別 context で呼び出し      <b>subagent / planner</b></p> <p>↓</p> <p>重</p>            | <p>軽</p> <p>即時に採点      <b>軽量 skill / LLM-as-judge</b></p> <p>別 context で PR 全体      <b>review agent</b></p> <p>仕様整合まで検査      <b>仕様レビュー agent</b></p> <p>↓</p> <p>重</p>      |

# ハーネスを足す前に考えるべき 4 ステップ

ゴール: ハーネスの変更容易性を高く保つ



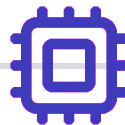
## 1. 業務目的を言語化

何を守る / 守らない  
/ 人が介入する  
+前提確認



## 2. 構造で守れな いか問う

設計 / 型 / 依存方向  
> scaffolding > test  
> 指示



## 3. 指示で補う

残りの CLAUDE.md  
/ skill / review  
agentは最小限



## 4. 削れる状態で 運用

小さく始める / 失敗  
駆動 / 目的で削る /  
ログを残す

**「ハーネスの前」 = エージェントの推論に載せずに済む可能性を潰してから、残ったものを載せる順序**

# 1. 業務目的を言語化する

目的が曖昧だと、制御は設計ではなく「不安への反応」になる

## 1 何を守るか？

複数ある。互いに衝突する

## 2 何を守らないか？

スコープ外

## 3 どこで人が介入するか？

エージェントに任せない判断

例: 「重い集計処理の軽量化」 (DWH 直叩き → マート層参照)

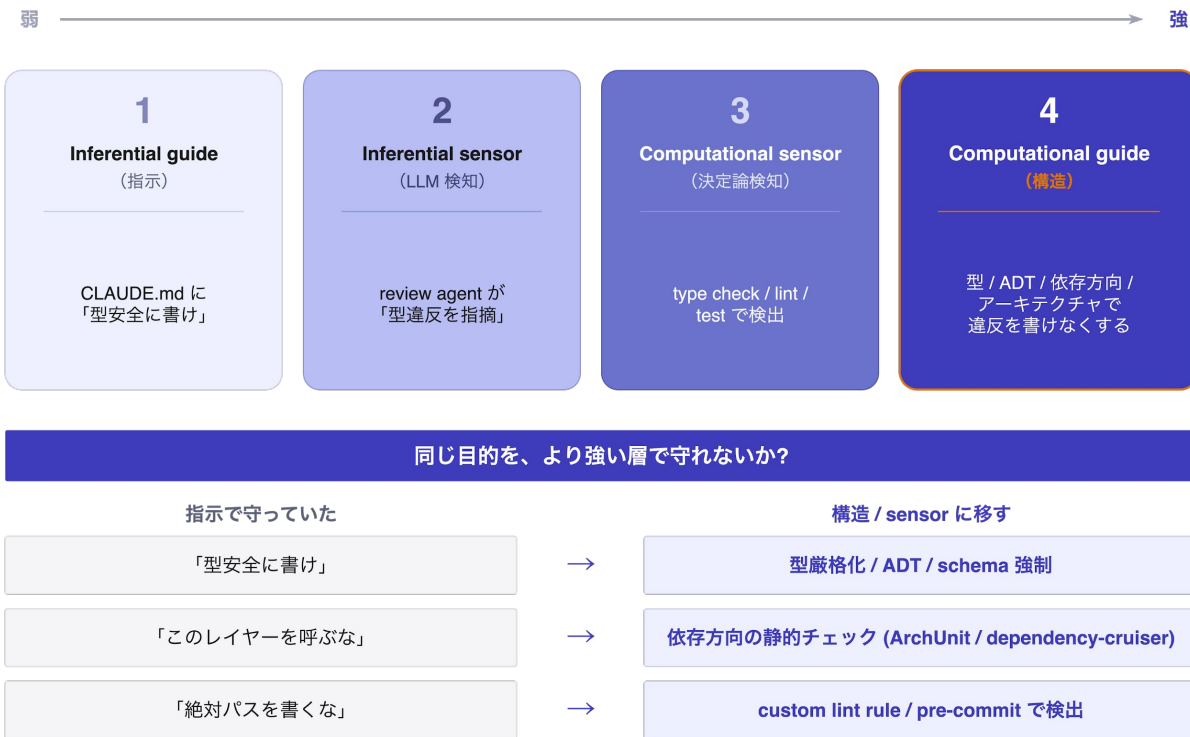
| 問い          | 答え                                          |
|-------------|---------------------------------------------|
| 何を守るか？      | 結果の数値一致 / 既存利用側 (ダッシュボード・API) 契約 / マート層スキーマ |
| 何を守らないか？    | 元クエリの可読性、最初からの最適設計、ドキュメントの同期                |
| どこで人が介入するか？ | 軽量化対象の優先順位、コスト/性能のトレードオフ、アーキ変更採否            |

前提として「Agentに任せられる業務か?」「開発環境はハーネスが効くか?」は満たす必要がある



## 2. エージェントではなく構造で守る

違反が書けない形にできるなら最強：構造で守れるなら、指示は要らない



### 3. 指示を足すのは簡単...判断基準は削るためにある

判断基準の本丸は「削るため」にある。ただし guide は評価しにくく、放っておくと溜まる。

同じ「削る判断」でも、見える情報がまったく違う



だから guide は最初から少なく保つ —— 足すハードルを上げる

## 4. 業務目的に照らして、既存ハーネスを棚卸しする

既存のハーネスを「判定」で動かす: 実務では次の3パターンが多い

|        | 既存要素                               | 判定後                         | なぜ                             |
|--------|------------------------------------|-----------------------------|--------------------------------|
| ① 観点絞る | 可読性 + セキュリティ + 数値妥当性の review agent | review agent は残す、可読性観点だけ落とす | 要素を消すのではなく、<br>スコープを絞る         |
| ② 層を移す | CLAUDE.md 「データ品質を担保せよ」(指示)         | rule → review agent の観点に移す  | 具体的チェック(数値一致・スキーマ)の方が強い        |
| ③ 目的次第 | 「対象クエリを Agent が自動選定」               | 軽量化プロジェクトでは不採用              | 業務目的が変われば採用<br>(例: ボトルネック自動発見) |

# 小さく始める、失敗駆動で育てる

実務ではセッション履歴だけは必ず保持した上で3・4から始めて失敗が積み重なってきたときに戻る方がスピード感持って進められる。

理想の順序

新規プロジェクトで取り組むときは構造をざっくり先に作ってから進める

業務目的を言語化

構造で守る

指示で補う

削れる状態で運用

実務だと...

業務目的を言語化

構造で守る

指示で補う

削れる状態で運用

曖昧さや指示に従わなくなって来た時にループ

必ずセッション履歴またはログは残すこと

# まとめ: 取り組む順序

## 1 業務目的を言語化

何を守る / 守らない / 人が介入する

## 2 構造で守れないか問う

設計 / 型 / 依存方向 / アーキテクチャ > scaffolding > test > 指示

## 3 指示で補う

基本は Skill (AGENTS.md) は最小限

## 4 削れる状態で運用

小さく始め、失敗駆動で足し、目的で削る

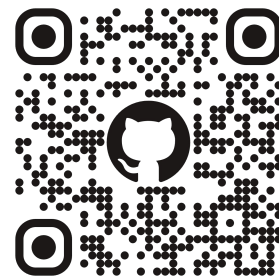
**判断基準を先に持つとシンプルかつ変更容易のハーネスを作りやすい**

---

# ありがとうございました！

質問やお問い合わせ等あればお気軽に連絡ください！

X: @gota\_bara



cc-sddのGithub

---